```
            T* old_data=nullptr;
            if(old_tail.ptr->data.compare_exchange_strong(     ⟵6
                  old_data,new_data.get()))
            {
                counted_node_ptr old_next={0};
                if(!old_tail.ptr->next.compare_exchange_strong(     ⟵7
                      old_next,new_next))
                {
                    delete new_next.ptr;          ⟵8
                    new_next=old_next;     ⟵9
                }
                set_new_tail(old_tail, new_next);
                new_data.release();
                break;
            }
            else          ⟵10
            {
                counted_node_ptr old_next={0};
                if(old_tail.ptr->next.compare_exchange_strong(     ⟵11
                      old_next,new_next))
                {
                    old_next=new_next;     ⟵12
                    new_next.ptr=new node;          ⟵13
                }
                set_new_tail(old_tail, old_next);     ⟵14
            }
        }
    }
};
```

This is similar to the original `push()` from listing 7.15, but there are a few crucial differences. If you *do* set the `data` pointer ❻, you need to handle the case where another thread has helped you, and there's now an `else` clause to do the helping ❿.

Having set the `data` pointer in the node ❻, this new version of `push()` updates the `next` pointer using `compare_exchange_strong()` ❼. You use `compare_exchange_strong()` to avoid looping. If the exchange fails, you know that another thread has already set the `next` pointer, so you don't need the new node you allocated at the beginning, and you can delete it ❽. You also want to use the `next` value that the other thread set for updating `tail` ❾.